

GitHub Pages: Manual Deployment

Understand which files are published before automating the process

This is practice; Project 2 uses GitHub Actions.

Learning Goals

- Explain which files GitHub Pages publishes.
- Read the directory layout of a manual deployment.
- Describe the repeated steps in manual deployment.

Before You Start

Finish these two lessons first:

- `githubio_installation.pdf` : create and clone your `yourusername.github.io` repository, and push with VS Code.
- `2. Getting Started with Hugo.pdf` : write your content, and set `baseUrl` in `hugo.toml` to `https://yourusername.github.io/`.

What GitHub Pages Hosts

GitHub Pages serves static HTML, CSS, JavaScript, and images.

- It does not execute Laravel/PHP or run MySQL.

So only the files that Hugo generates in `public/` are published.

Your repository name and website address are explained in

`githubio_installation.pdf` .

Your Directory Layout

Work in one directory that contains three items in this example:

```
├── my-portfolio/           ← Hugo source: you edit this
│   ├── content/
│   ├── layouts/
│   ├── static/
│   ├── archetypes/
│   └── hugo.toml
├── yourusername.github.io/ ← cloned repository: GitHub Pages publishes this
│   ├── .git/
│   ├── README.md
│   ├── index.html         ← first page from the installation lesson
└── run.sh                 ← builds my-portfolio/
```

`my-portfolio/` and `yourusername.github.io/` are **separate** directories.

Step 1: Build the Site

From the directory that contains `run.sh` :

```
./run.sh
```

Hugo creates `my-portfolio/public/` . Only `public/` is new:

```
my-portfolio/
├── content/  layouts/  static/  archetypes/  hugo.toml
├── public/
│   ├── index.html
│   ├── about/
│   ├── projects/
│   ├── posts/
│   ├── css/
│   └── images/
└──
```

← NEW: generated website files

Step 2: Copy the Built Files

From the same directory, copy the **contents** of `public/` into the repository:

```
cp -R my-portfolio/public/. nkuase.github.io/  
touch nkuase.github.io/.nojekyll
```

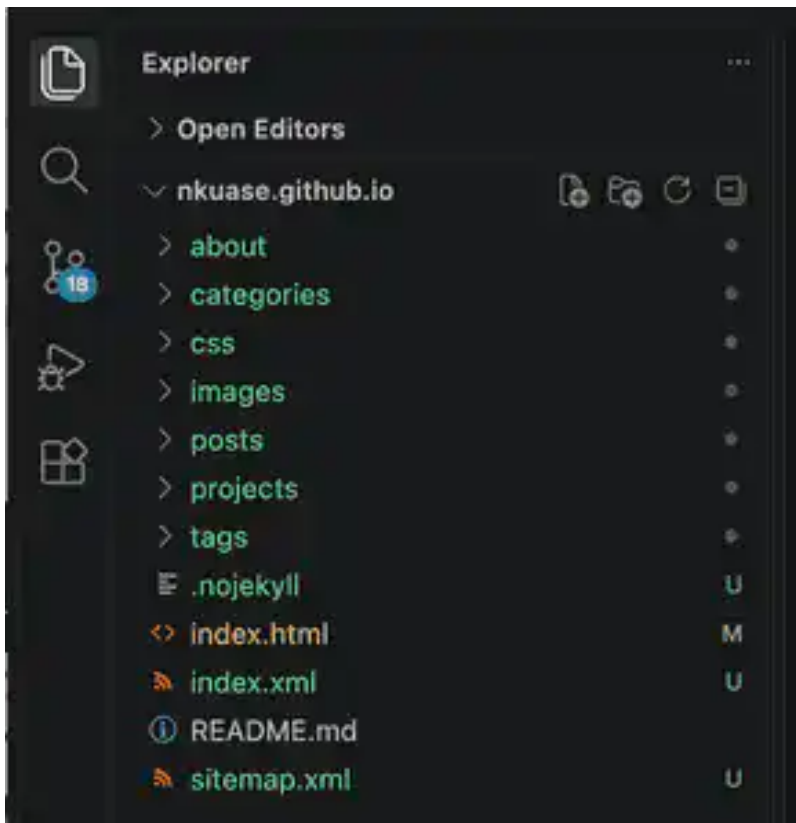
The repository now holds the generated website:

```
yourusername.github.io/  
├── .git/  
├── .nojekyll  
├── README.md  
├── index.html  
├── about/  
├── projects/  
├── posts/  
├── css/  
└── images/
```

← NEW: serve files as they are
← replaced by Hugo's home page
← copied from public/

```
cp -R my-portfolio/public/. nkuase.github.io/  
touch nkuase.github.io/.nojekyll
```

- VSCode detects the change.



Copy the Contents, Not the Folder

`public/.` copies what is **inside** `public/`.

If you copy `public` itself, the home page ends up at `yourusername.github.io/public/index.html`, and the website address no longer works.

`.nojekyll` tells GitHub Pages to skip Jekyll processing and serve these files as they are.

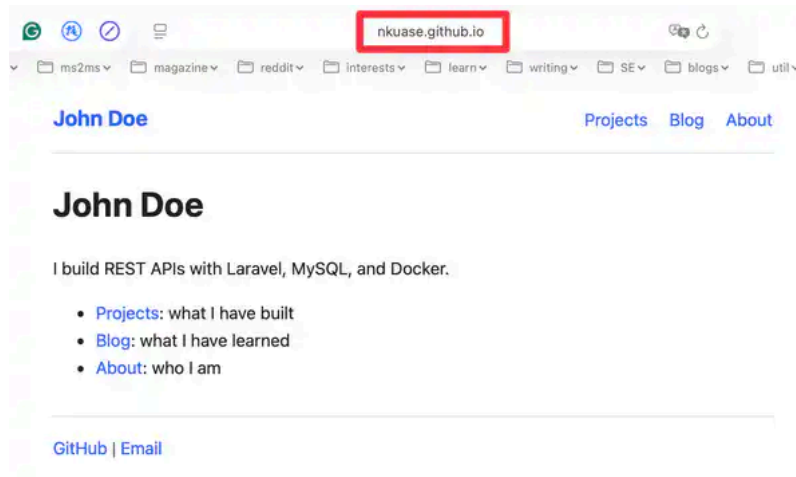
Keep your Hugo source in `my-portfolio/`, outside the repository.

Step 3: Publish and Verify

Open the `yourusername.github.io` folder in VS Code.

- Publish the copied files the same way as your first page: add, commit, and push, and then check the result (Steps 4 to 6 of `githubio_installation.pdf`).

Open the published URL and check navigation and screenshots.



Why Automate This?

Manual updates repeat three steps: build, copy, push.

Files you delete from `my-portfolio/` also stay in the repository, because `cp` only adds files.

- GitHub Actions can build and publish from your Hugo source.
- For Project 2, switch Pages to **GitHub Actions** and follow lessons 2 to 4.
- Do not keep using the manual-copy procedure after that switch.